

Smart MCQ Solver Report

Objective

The goal of this project is to build and select a deep learning model by using the different basics Machine learning, Natural Language Processing (NLP) and Deep learning approaches and evaluating these approaches using the MAP@3 metric, capable of predicting and ranking the top three most probable answers for a given question.

Abstract

This competition has a goal of creating a Multiple Choice Question Answering where the models requires to understand the context of a question and accurately rank the most appropriate answer choices along with two options which also has a good probabilistic chance from five candidate options for each question using the Mean Average Precision at 3 (MAP@3) metric.

The project begins with the detailed explanatory of the data analysis (EDA) to understand the characteristics, followed by the preprocessing techniques which includes label encoding, text data validation, text cleaning and mcq formatting (for which we created question and option pairs) designed to preserve semantic information and later multiple models were tested including tf-idf using cosine similarity, a scratch model built using BiLSTM and attention layer and a pretrained model RoBERTa which is a pretrained transformer-based multiple-choice model. Each approach is implemented, evaluated and compared using MAP@3 score and wandb performance graphs to easily visualize the performances and have a clear idea.

After creating and testing the models we get the experimental results which shows that traditional tf-idf cosine similarity shows the worst score with accuracy of 0.093 and the best model which is the BiLSTM with attention layer clearly dominates both the accuracy and MAP@3 score. The final competition submission achieved a Kaggle MAP@3 score of 0.74563, successfully exceeding the minimum qualification threshold.

While the project shows the importance of contextual language understanding is essential for mcq answering, traditional lexical methods provide a useful baseline, deep learning approaches particularly the custom BiLSTM with attention and RoBERTa model - achieve substantially better performance by learning semantic relationships between question and answer options.

Dataset and Preprocessing

- Dataset Description

This competition contains three datasets one is train.csv (contains the training questions and along with the correct answers), test.csv(contains the test questions without the answers and we need to generate the questions) and a sample_submission.csv (contains the format to submit in the competition). Upon using pandas(read_csv function which helps to read the comma separated values) we loaded the dataset and checked for the shapes of the each dataset which is for train.csv its (2000,8) and for test.csv its (500,7).

The train and test dataset have the same features which are id, prompt or question, A, B, C, D and E (each question has five candidate answer options A to E) while the train dataset has another feature which is the answer upon which we need to train our models.

train.info()					test.info()				
#	Column	Non-Null Count	Dtype		#	Column	Non-Null Count	Dtype	
0	id	2000 non-null	int64		0	id	500 non-null	int64	
1	prompt	2000 non-null	object		1	prompt	500 non-null	object	
2	A	2000 non-null	object		2	A	500 non-null	object	
3	B	2000 non-null	object		3	B	500 non-null	object	
4	C	2000 non-null	object		4	C	500 non-null	object	
5	D	2000 non-null	object		5	D	500 non-null	object	
6	E	2000 non-null	object		6	E	500 non-null	object	
7	answer	2000 non-null	object		dtypes: int64(1), object(6)				
dtypes: int64(1), object(7)									

An example row of train and test dataset -

```
display(train_cc.head())display(test_cc.head())
```

Gives –

	id	prompt	A	B	C	D	E	answer
0	1	Pick the best possible answer: What is Martin ...	Martin Heidegger believes that humans exist wi...	Martin Heidegger believes that humans do not e...	Martin Heidegger does not believe in the exist...	Martin Heidegger believes that the relationshi...	Martin Heidegger believes that time is an illu...	B

And for test

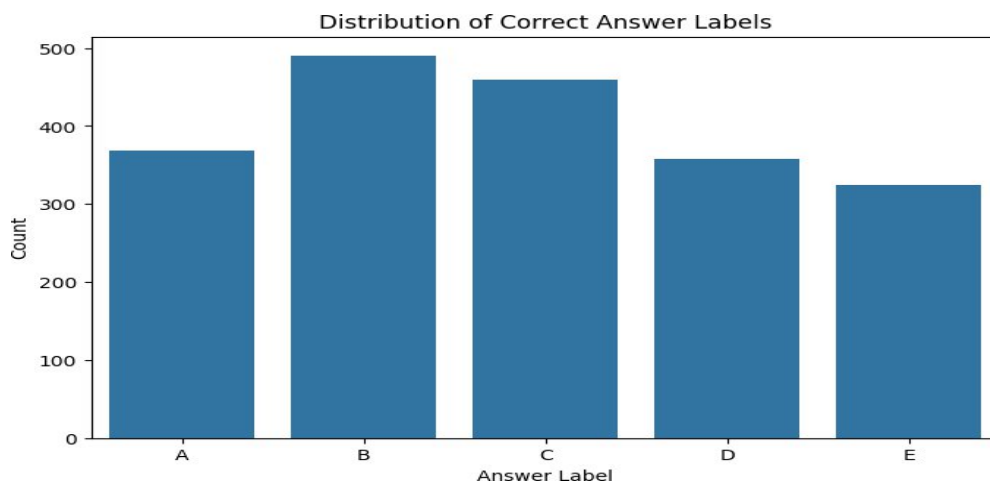
	id	prompt	A	B	C	D	E
0	1	Pick the best possible answer: What is the rel...	For every eigenstate of one Hamiltonian, its p...	For every eigenstate of one Hamiltonian, its p...	For every eigenstate of one Hamiltonian, its p...	For every eigenstate of one Hamiltonian, its p...	For every eigenstate of one Hamiltonian, its p...

- Exploratory Data Analysis

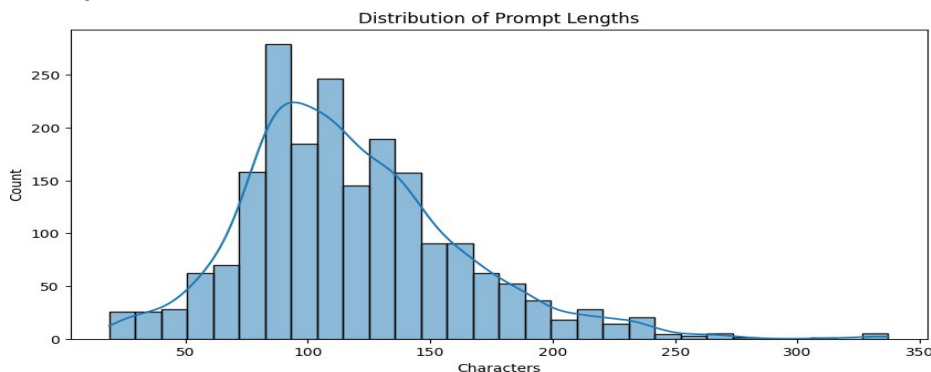
Using `train.isnull().sum()` confirms that there is **no missing data** in the train dataset so there is **no imputation required** in the dataset.

While `train_duplicates = train.drop(columns = ["id"]).duplicated().sum()` which gives us that there are **183 rows are duplicate** where question prompt, all five options and the correct answers are identical differing only in the ID field. Since these duplicates are part of the original dataset **they were retrained during model development to preserve the distribution**. While preliminary experiments were conducted with and without the duplicate removal which shows that retraining the original dataset produces better results. As the rows are fully duplicate of one another for which they do not cause any **label ambiguity** while repeated samples reinforce the same question-answer relationships during training allowing the models to learn these patterns more effectively.

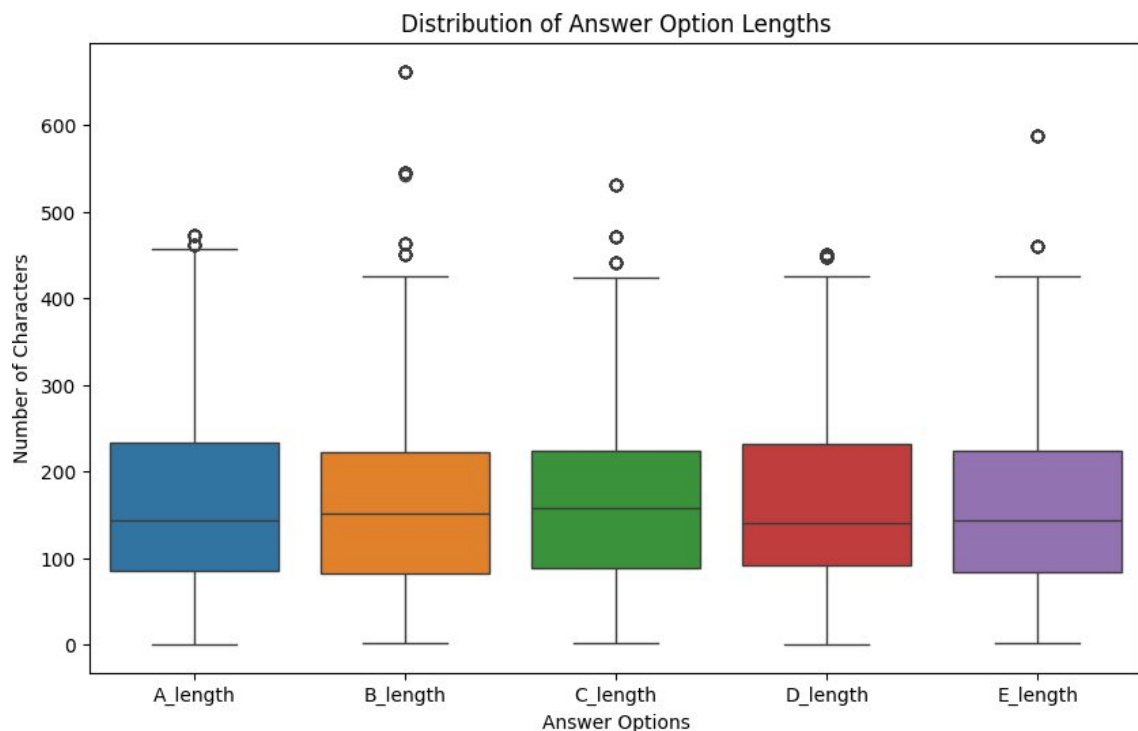
While checking the target class distribution we found that **option B and C occurs most** while the **option E occurs the least**. Although the dataset is not perfectly balanced the class imbalance is relatively small so it's unlikely to require any additional balancing such as **oversampling** or **class weighting**.



After plotting the prompt length distribution graph we can see that average prompt length is 118 characters where most questions fall within 80-150 characters where some questions extend beyond 250 characters producing a slight right skew distribution. Since the prompt lengths remain within a manageable range so sequence truncation (used in BiLSTM and RoBERTa) is unlikely to remove substantial information.



After plotting the option length distribution we see that average lengths of answer options ranging from 162 to 167 characters. Since no options are consistently longer or shorter, the model is less likely to learn superficial biases based solely on answer lengths.



So after performing EDA we can conclude that dataset is clean, well-structured containing no missing values and with only moderate no. of duplicate questions. The target labels are balanced across all five answer classes which reduces the risk of severe class imbalance during training.

• Data Preprocessing

The four preprocessing methods I have implemented in the training dataset are

1. Label Encoding

In the given original dataset the correct answers are represented using categorical labels A to E but the deep learning algorithms or machine learning algorithms cannot directly optimize categorical string values so we converted the target variable into numerical labels using LabelEncoder.

```
label_encoder=LabelEncoder()
train_cc["label"]=label_encoder.fit_transform(train_cc["answer"])
mapping=pd.DataFrame({
    "Answer Label":label_encoder.classes_,
    "Encoded Label":range(len(label_encoder.classes_))
})
display(mapping)
```

The resulting mapping is shown below.

	Answer Label	Encoded Label
0	A	0
1	B	1
2	C	2
3	D	3
4	E	4
...		

Label Encoding does not introduce any ordinal relationship between the answer classes. Although the labels are represented numerically as 0–4, these values merely serve as identifiers for the five mutually exclusive classes and are not interpreted as ordered quantities by the classification models.

label encoding defines one to one mapping $f: Y \rightarrow \{0, 1, 2, 3, 4\}$ thus

$f(A) = 0, f(B) = 1, f(C) = 2, f(D) = 3, f(E) = 4.$

2. Text Data Validation

In this preprocessing step we have explicitly converted all the question prompts and answer options into string format before further processing. This eliminates the inconsistencies caused by the unexpected data type during text processing although the columns were already stored as object data types.

This step guarantees that every textual input can be processed uniformly by the models.

```
train_cc[text_columns] = train_cc[text_columns].astype(str)
```

3. Text Cleaning

In the text cleaning we have n't removed or modified punctuations, capitalization and sentence structure as modern contextual languages rely heavily on these parts. By removing these linguistic cues often degrades performance.

The Operations done in the text cleaning are

Operations	Purpose
Remove leading/trailing spaces	Standardize formatting
Replace multiple spaces	Improve consistency
Replace newline characters	Produce continuous text
Replace tab characters	Remove formatting artifacts

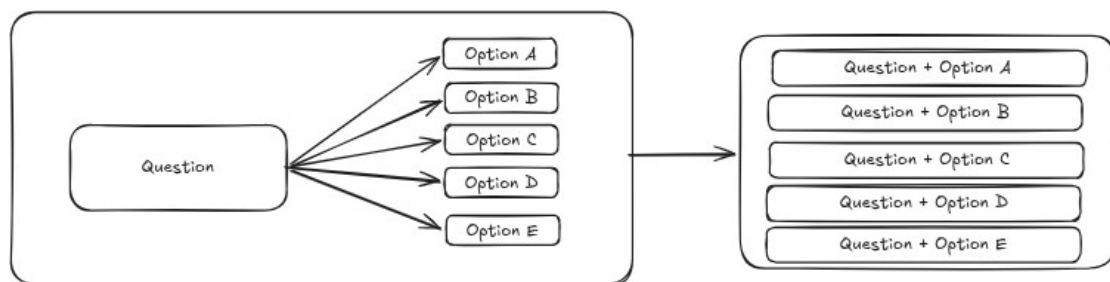
4. MCQ Formatting

As during EDA and dataset exploration we find that each question prompt contains five answer options so instead of processing the question independently, every answer option is combined with the corresponding question to form a question-option pair.

This formatting generates the following

Question: <question text>

Answer: <candidate option>



Summary for the Data preprocessing –

Step	Purpose
Label Encoding	Convert categorical labels into numerical form
Text Validation	Ensure all text fields are stored consistently
Text Cleaning	Normalize whitespace while preserving semantics
MCQ Formatting	Construct Question–Option pairs for model input

The performed preprocessing avoids any heavy linguistics transformations to preserve the semantic and contextual information present in the original dataset.

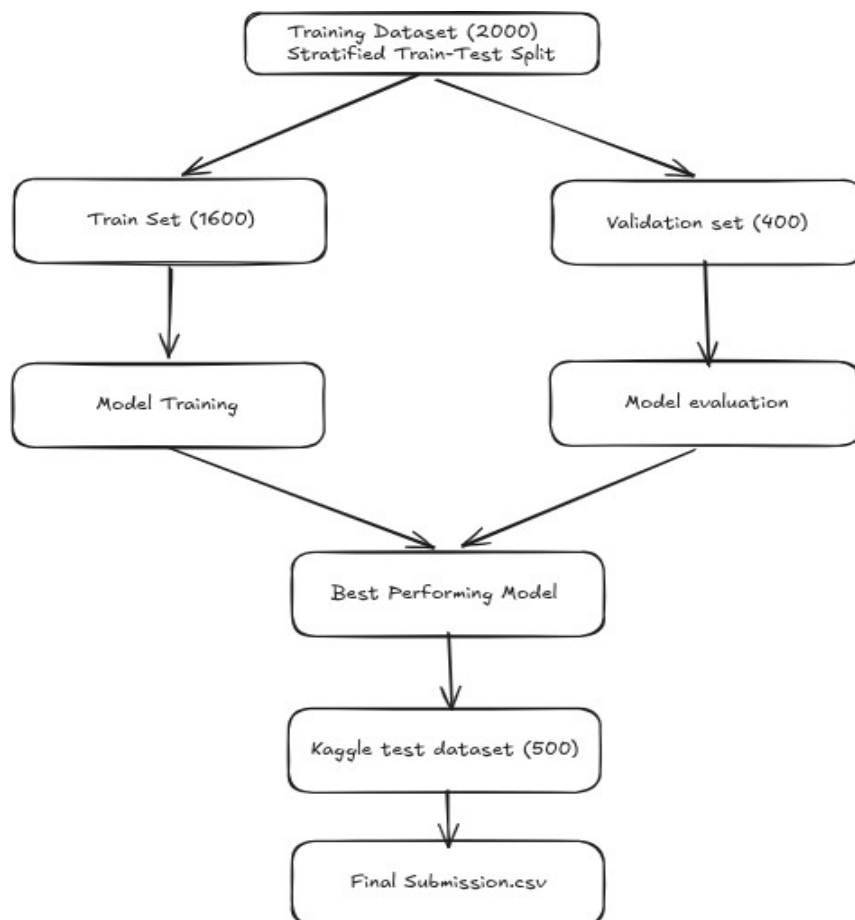
Train-Validation Split

After preprocessing the dataset was partitioned into training and validation subsets to enable an unbiased evaluation of the developed models.

The training subset was used for learning model parameters, whereas the validation subset used for model comparison and performance evaluation.

We have splitted the dataset into 80:20 where training set contains 80%(1600 samples) and 20%(400 samples) of validation set. Since the target labels are not perfectly balanced a stratified split is used to preserve approximately the same class distribution in both the training and validation. A random state or random seed of 42 is used which guarantees

that the dataset is partitioned identically across multiple executions allowing fair comparisons between the models.



• Model Evaluation Metrics

To compare all developed models fairly, the same evaluation metrics are used which are Accuracy, Macro F1-score and Mean Average Precision at 3 (MAP@3) on the validation dataset.

1. Accuracy

Accuracy measures the proportion of validation samples whose highest ranked prediction matches the ground-truth answer.

$Accuracy = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$ although accuracy is simple to interpret it only considers the first predicted answer and ignores whether the correct answer appears in the remaining ranked predictions.

2. Macro F1 -Score

Macro F1-score evaluates classification performance equally across all five answer classes (A–E).

$$Macro\ F1 = \frac{1}{C} \sum_{i=1}^C F1_i$$

It first computes F1 scores for each class then averages them.

3. Mean Average Precision at 3 (MAP@3)

Each model predicts three ranked answer options. For one question

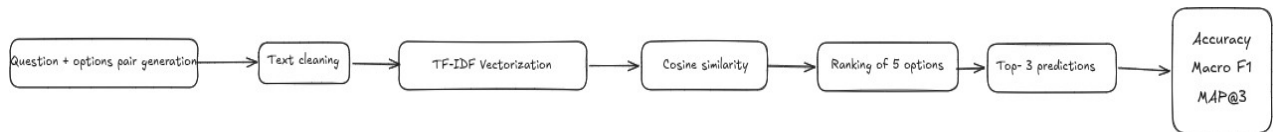
$$AP@3 = \begin{cases} 1 & \text{Correct answer ranked first} \\ 1/2 & \text{Correct answer ranked second} \\ 1/3 & \text{Correct answer ranked third} \\ 0 & \text{Otherwise} \end{cases}$$

Then we compute the mean by using $MAP@3 = \frac{1}{N} \sum_{i=1}^N AP_i@3$

TF-IDF (Baseline Model)

The first model is a TF-IDF (Term Frequency-Inverse Document Frequency) based retrieval model. As we know TF-IDF doesn't learn contextual or semantic representations from data, instead it represents every document as a sparse vector based on the statistical importance of each word.

The pipeline I have used for the TF-IDF Baseline model is shown below



First we know that $TF(t, d) = \frac{f_{t,d}}{\sum_i f_{i,d}}$ and

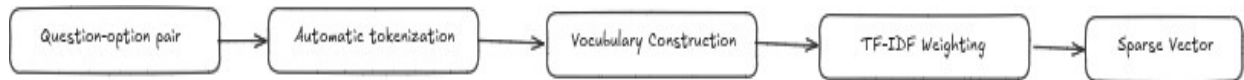
for the IDF we know $IDF(t) = \log\left(\frac{N}{df_t}\right)$

then TF-IDF equation becomes $TFIDF(t, d) = TF(t, d) \times IDF(t)$

The first phase of our pipeline is generating question option pairs by using the same function of `create_question_option_pair` created in the data preprocessing with pairing question + answer+ options.

Before vectorization every question-option pair is passed through the text cleaning function where only whitespace normalization is performed.

TF-IDF vectorizer is then fitted only on the training corpus to avoid information leakage from the validation dataset. During fitting, the vectorizer automatically tokenizes the text, constructs a vocabulary, and computes TF-IDF weights for every token. Each Question–Option pair is therefore represented as a sparse numerical vector in the vocabulary space.



Once both the question and question-option pair are converted into TF-IDF vectors we computed similarity using cosine similarity $\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$

After computation of the similarity score independently for all five candidate answers, these scores are sorted in descending order.

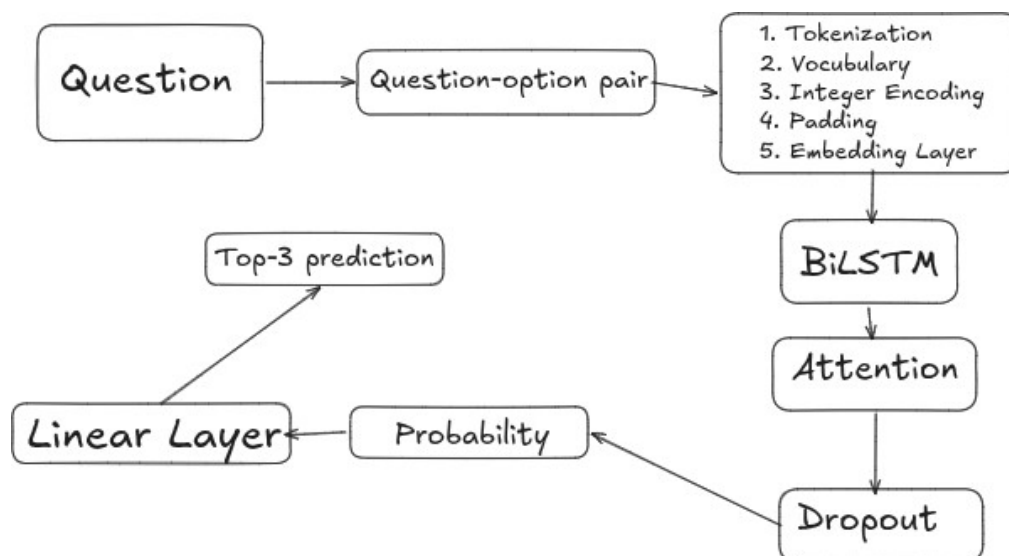
By applying the performance evaluation on the validation set we get Accuracy of 0.0950, Macro F1 of 0.0921 and MAP@3 of 0.2175.

Although its performance is considerably lower than the deep learning models, it establishes a useful baseline for comparison. The primary limitation of TF-IDF is that it relies solely on lexical overlap and cannot capture contextual meaning or semantic relationships between words.

BiLSTM + Attention

This is the Scratch model which is developed entirely from scratch using pytorch which combines a Bidirectional Long Short-Term Memory (BiLSTM) network with an Attention mechanism to learn contextual representations of Question–Option pairs. Instead of treating the task as a five-class classification problem, the model predicts the relevance of each Question–Option pair independently and ranks all candidate options according to their predicted probabilities.

The pipeline of the model is shown



The original dataset contains one question associated with five candidate answers and one correct option. So we transformed it into a binary relevance prediction task which increases the number of training samples from **1,600 questions to 8,000 Question–Option pairs**, enabling the model to learn whether an individual answer option correctly matches the corresponding question.

For the tokenization a simple whitespace tokenizer is used to split each sentence into tokens (what is AI? → [“what”, “is”, “ai”]). This lightweight approach is sufficient for a baseline deep learning model while keeping the preprocessing pipeline transparent and easy to interpret.

After tokenization, the vocabulary is constructed from the training dataset. A frequency-based dictionary is created by counting the occurrence of every token using Python's counter class. To handle special cases during training and inference two reserved tokens are added

Token	Index	Purpose
<PAD>	0	Used to pad sequences to a fixed length
<UNK>	1	Represents words that are not present in the vocabulary

After constructing the vocabulary, each token is converted into its corresponding integer index. This process transforms textual data into a numerical representation that can be processed by the deep learning model.

Tokenization → Vocabulary Lookup → Integer encoding

Neural networks process data in mini-batches, where all input sequences must have the same length. Since the lengths of the Question–Option pairs vary, a fixed maximum sequence length of **64 tokens** is selected. If a sequence contains **fewer than 64 tokens**, the remaining positions are filled with the <PAD> token (index 0). If a sequence contains **more than 64 tokens**, it is truncated to the first 64 tokens.

Before training the model, the encoded sequences are converted into a custom PyTorch Dataset.

Each training sample contains

- an encoded Question–Option sequence
- its corresponding binary label (0 or 1)

The custom Dataset allows PyTorch to efficiently load data during training while keeping the preprocessing pipeline independent from the model implementation.

After creating we created a dataloader where the dataset is divided into mini-batches using PyTorch DataLoader. The batch size is selected as 32 which produces batches of shape (32,64).

The numerical token IDs cannot be processed directly by recurrent neural networks. Therefore, an embedding layer is used to transform every token into a dense continuous vector representation.

If x_i denotes the token index, then the embedding layer learns $e_i = E(x_i)$ where E is the embedding matrix and e_i is the learned embedding vector.

The embedded sequence is passed through a Bidirectional Long Short-Term Memory (BiLSTM) network. Unlike a standard LSTM, BiLSTM processes the sequence in both forward and backward directions, enabling the model to capture information from both past and future words.

For each time step $\vec{h}_t$ represents forward hidden state while $\overleftarrow{h}_t$, the final hidden representation becomes $h_t = [\vec{h}_t; \overleftarrow{h}_t]$ where $;$ denotes vector concatenation.

Now not every word contributes equally to selecting the correct answer. Therefore, an attention mechanism is introduced after the BiLSTM layer. First, an attention score is computed $e_t = Wh_t + b$ then the scores are normalized

using $\text{softmax}(\alpha_t)$ then finally context vector is calculated $c = \sum_t \alpha_t h_t$. The

attention weights allow the model to focus more on informative words while reducing the influence of less important tokens.

To reduce overfitting, a dropout layer with probability $p=0.3$ is applied to the attention output. During training, dropout randomly deactivates a subset of neurons, encouraging the model to learn more robust feature representations.

The context vector is finally passed through a fully connected linear layer $z = Wc + b$ which produces a single prediction score (logit) for every Question–Option pair. These scores later grouped together to rank the five options.

The model is trained using Binary Cross Entropy with Logits Loss (BCEWithLogitsLoss).

The loss combines the sigmoid activation and binary cross entropy into a numerically stable formulation.

Its objective is $L = -y \log(\sigma(z)) - (1 - y) \log(1 - \sigma(z))$.

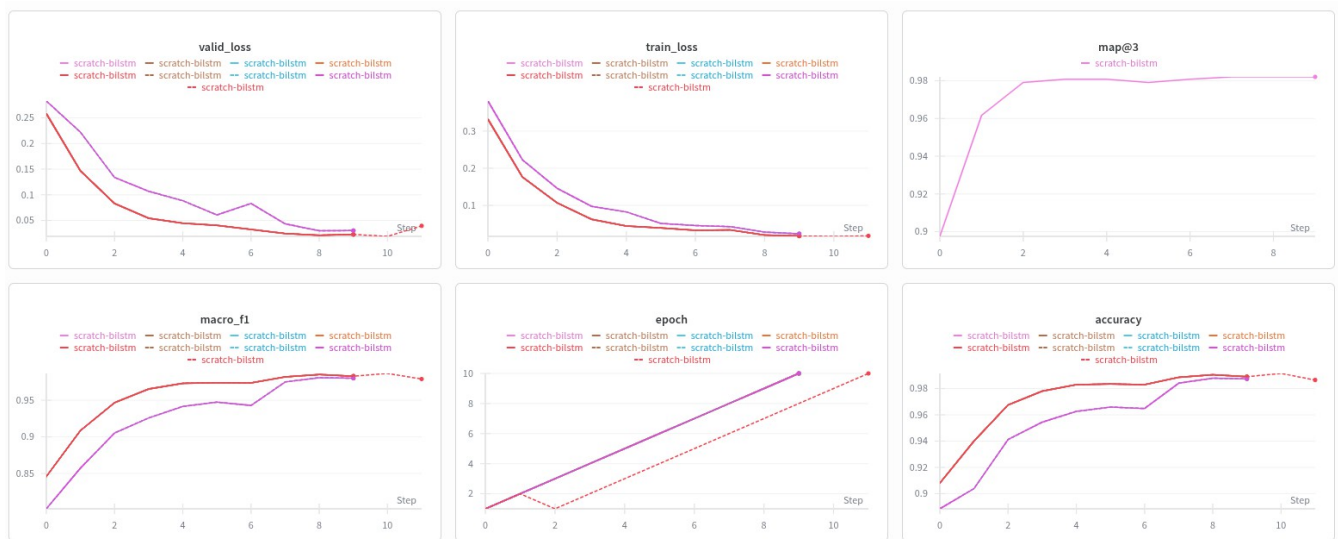
The network parameters are optimized using the Adam optimizer with a learning rate of 10^{-3} .

The Training Pipelines is shown below



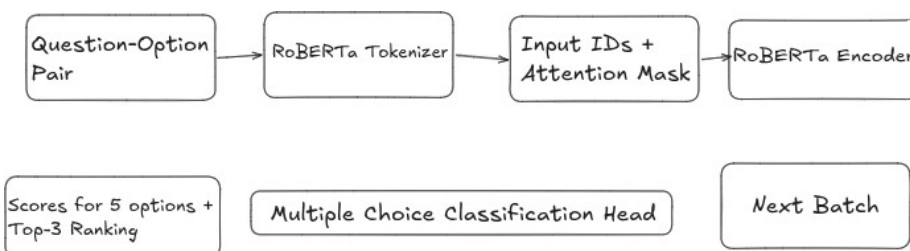
Now after every epoch, the model is evaluated on the validation dataset. The validation phase does not update the model parameters; instead, it measures the generalization performance using validation loss, accuracy, Macro F1 Score, MAP@3.

The best model scores MAP@3 0.9821, Macro F1 is 0.9850 and accuracy 0.989



Pretrained RoBERTa

The third model uses RoBERTa (Robustly Optimized BERT Pretraining Approach), a pretrained transformer language model developed by Facebook AI. Unlike the scratch BiLSTM model, RoBERTa has already learned rich contextual language representations from large-scale text corpora. Instead of training a language model from scratch, the pretrained encoder is fine-tuned for the multiple-choice question answering task.



Unlike the scratch BiLSTM model, RoBERTa does not use a manually created vocabulary. Instead, the pretrained RoBERTa tokenizer automatically performs subword tokenization, converts every token into its corresponding vocabulary index, and generates an attention mask indicating valid tokens and padding positions. Where we used `max_length = 128`, `truncation = true` and padding to `max_length`. Every Question–Option pair is therefore converted into two tensors: Input IDs and Attention Mask.

Unlike the BiLSTM model, which predicts every Question–Option pair independently, the RoBERTa model processes all five candidate answers simultaneously.

Question –> Options A to E –> Tokenizer –> (5 × 128) –> RoBERTa

The pretrained `roberta-base` model is loaded using Hugging Face's `AutoModelForMultipleChoice`. The model already contains millions of pretrained parameters learned from large-scale unlabeled text corpora. During fine-tuning, only the model parameters are updated to adapt the pretrained language representations to the MCQ task.

For the Fine tuning we used Epochs = 3, Batch Size = 8, Learning Rate = 2×10^{-5} and Weight Decay = 0.01. Instead of training the network from scratch, only fine-tuning is performed. Fine-tuning updates the pretrained parameters using the project dataset while preserving the rich language knowledge learned during pretraining.

Hugging Face's Trainer API is used to simplify the training pipeline by automatically managing forward propagation, backpropagation, evaluation after every epoch, checkpoint saving, and model selection.

`load_best_model_at_end=True` and `metric_for_best_model="macro_f1"`

Epoch	Training Loss	Validation Loss	Accuracy	Macro F1
1	2.996464	2.269812	0.640000	0.637711
2	1.746970	1.053925	0.867500	0.868409
3	1.188556	0.907776	0.882500	0.883625

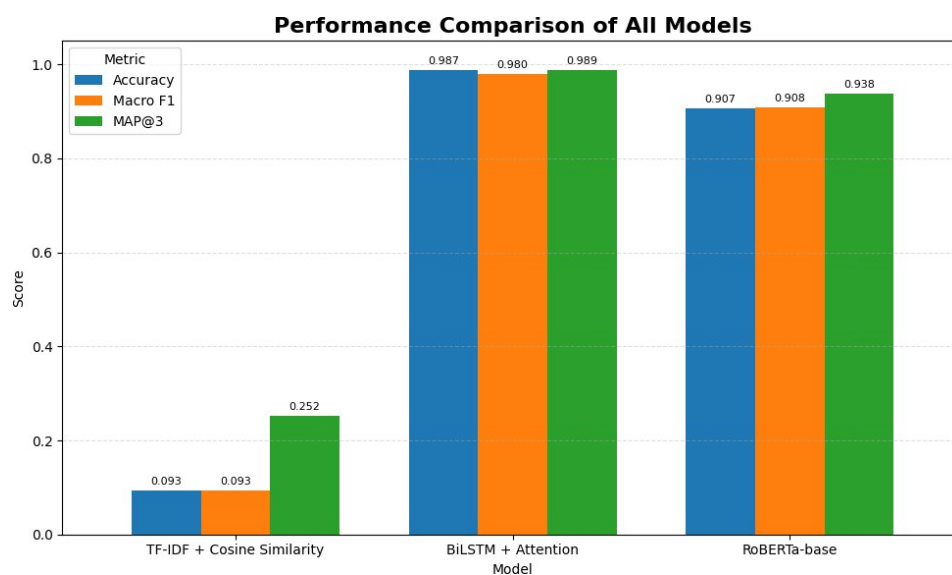
The validation metrics consistently improved across epochs, indicating that the pretrained language representations successfully adapted to the multiple-choice question answering task. The final MAP@3 score for RoBERTa is 0.9217.



Performance & Comparative Analysis

Three different approaches were implemented and evaluated on the Smart MCQ Solver Challenge dataset. These models represent three distinct paradigms for solving the multiple-choice question answering problem.

Model	Type	Accuracy	Macro F1	MAP@3
TF-IDF + Cosine Similarity	Traditional Baseline	0.0934	0.0929	0.2518
BiLSTM + Attention	Scratch Deep Learning	0.9874	0.9800	0.9886
RoBERTa-base	Pretrained Transformer	0.9066	0.9076	0.9382



The bar chart clearly illustrates the performance differences among the three approaches.

The **TF-IDF baseline** achieves the lowest performance across all evaluation metrics. Since TF-IDF relies purely on lexical overlap between the question and answer options, it cannot capture contextual meaning or semantic relationships.

The **BiLSTM with Attention** model achieves the highest performance on all three evaluation metrics. By learning contextual representations in both forward and backward directions and using an attention mechanism to emphasize the most informative words, the model effectively captures semantic relationships between questions and answer options.

Although **RoBERTa-base** is a powerful pretrained transformer model, its performance in this project remains slightly below that of the scratch BiLSTM model. This may be attributed to the relatively small dataset size, the limited number of fine-tuning epochs, and computational constraints during training.

Comparison	MAP@3 Improvement
BiLSTM vs TF-IDF	+0.7368
RoBERTa vs TF-IDF	+0.6864
BiLSTM vs RoBERTa	+0.0504

So the final Scratch BiLSTM model is kept as the best model and used to generate predictions for the unseen test dataset and prepare the final competition submission. Which then scores 0.74563 and gets me to rank 958 in the competition.

Challenges Faced

1. Designing a Scratch Deep learning model which includes tokenization, vocabulary construction, numerical encoding, padding, embedding, the BiLSTM architecture, and the attention mechanism.
2. Multiple-choice Data Representation which involves designing an effective representation suitable for different models required transforming the dataset into Question–Option pairs while preserving the relationship between each question and its corresponding answer choices.

Key Findings

- Traditional lexical retrieval methods such as TF-IDF are computationally simple but fail to capture contextual relationships between questions and answer options, resulting in comparatively low performance.
- Converting each multiple-choice question into independent Question–Option pairs provides an effective representation that can be used consistently across the models
- Although RoBERTa benefits from large-scale pretraining and achieves strong performance with relatively little preprocessing, the custom BiLSTM with Attention achieved superior performance on this dataset.
- The MAP@3 metric provides a more informative evaluation than simple classification accuracy because it rewards correctly ranking the answer within the top three predictions, which closely matches the competition objective.

Conclusion and Key Findings

This project successfully explored three different approaches for solving the Smart MCQ Solver Challenge, ranging from a traditional information retrieval baseline to custom deep learning and pretrained transformer-based models. Each approach demonstrated different strengths and limitations, providing valuable insight into how model complexity influences performance on multiple-choice question answering tasks.

The final Kaggle submission achieved a **MAP@3 score of 0.74563**, successfully exceeding the competition qualification threshold and validating the effectiveness of the developed pipeline.

The project also provided several practical learnings:

- Understanding the importance of exploratory data analysis before model development.
- Designing an effective preprocessing pipeline while preserving semantic information.
- Building a complete deep learning architecture from scratch using PyTorch.
- Learning how Bidirectional LSTM and Attention mechanisms improve contextual language modeling.

Resources

- PyTorch. (2026). *PyTorch Documentation*.
- Scikit-learn Documentation
- Hugging face and their models